Full GUI Upgrade Specification for Cerebro (Blueprint V2.0) — v2, corrected

This is an **incremental shared-screen upgrade specification**, not a tested patch. AI Studio must first verify the actual renderer, component paths, tokens and state adapters in the ZIP, then adapt these examples to the installed versions. Preserve the existing architecture — do not migrate to R3F merely for visual polish.

Target: the blueprint's shared screen — persistent center Cerebro widget, left spatial stations, right active surface, bottom health strip — plus the visual glossary effects: transmission glass sphere, Fresnel rim, fine orbital loops, drifting particles, threshold-based bloom, reduced-motion support.

What was fixed from the v1 draft

1. **The Fresnel shader patch compiled nothing** — it replaced `'#include '` (no chunk name) with an unbalanced block. v2 targets `<opaque_fragment>` (three r154+ name) with a guard that warns instead of silently skipping.
2. **The widget lost its JSX and two of its five axes** — the blueprint requires connection / input / output / execution / **persistence**. All five restored.
3. **Particle drift accumulated** — `pos.setY(pos.getY() + …)` compounds every frame. v2 computes bounded offsets from immutable base positions.
4. **The orbital "arcs" were not closed** — `EllipseCurve.getPoints()` repeats the endpoint and `CatmullRomCurve3` defaults to open. v2 drops the duplicate and closes the curve.
5. **"Selective bloom" was overstated** — a threshold affects any bright pixel; v2 calls it threshold-based bloom and treats the numbers as proposed tuning.
6. **The responsive CSS overflowed** — fixed tablet coordinates were applied across an 820–1200px range, the 481–819px range had no composition, labels went below 12px, and a 44px button sat in a 40px strip. v2 uses a gap grid and treats the blueprint's 1024×768 coordinates as a reference composition.
7. **No disposal, no live reduced-motion** — v2 returns `dispose()` and takes the reduced-motion preference per frame, so a preference change mid-session is respected.

1. Architecture Overview

```
App
├── ExperienceCanvas / DualSphereCanvas (3D world + sphere)
│   └── CerebroSphere (glass + rim + orbital loops + particles) + Stations + Bloom
├── PersistentCerebroWidget (screen-stable center overlay — 5-axis state)
├── SpatialStationsWorld (left elevated stations)
├── ActiveSurfacePanel (right — one project or addressed member, scrollable)
```

```
├── ProjectHealthStrip (bottom, always visible)
└── Existing: VoiceCard, PanelsOverlay, GuiFocusOverlay, data layer
```

2. Enhanced Cerebro Sphere (glass + Fresnel rim + orbital loops + particles)

```
// src/three/CerebroSphere.ts  (adapt the path to the project)
import * as THREE from 'three';

export interface CerebroSphere {
  group: THREE.Group;
  material: THREE.MeshPhysicalMaterial;
  points: THREE.Points;
  /** Call each frame from the EXISTING render loop. Pass the CURRENT
      reduced-motion preference so mid-session changes are respected. */
  update: (delta: number, reducedMotion: boolean) => void;
  dispose: () => void;
}

export function createCerebroSphere(scene: THREE.Scene): CerebroSphere {
  const group = new THREE.Group();
  const disposables: Array<{ dispose: () => void }> = [];

  /* ── Core glass sphere ── */
  const geometry = new THREE.SphereGeometry(6.2, 64, 64);
  const material = new THREE.MeshPhysicalMaterial({
    color: 0x4C8DFF,
    metalness: 0.1,
    roughness: 0.08,
    transmission: 0.92,
    thickness: 1.4,
    ior: 1.45,
    clearcoat: 0.6,
    clearcoatRoughness: 0.05,
    attenuationColor: new THREE.Color(0x4C8DFF),
    attenuationDistance: 2.5,
    emissive: new THREE.Color(0x1a3a6e),
    emissiveIntensity: 0.35,
  });
  disposables.push(geometry, material);

  /* ── Fresnel rim via onBeforeCompile.
     three r154+ renamed <output_fragment> to <opaque_fragment>.
     `normal` and `vViewPosition` are in scope at this point in the
     physical fragment shader. If the chunk is missing, warn — never
     silently skip. ── */
  material.onBeforeCompile = (shader) => {
    const target = '#include <opaque_fragment>';
    if (!shader.fragmentShader.includes(target)) {
      console.warn(
        `[CerebroSphere] ${target} not found (three r${THREE.REVISION}). ` +
        'Fresnel rim skipped — check the installed three version.'
      );
      return;
    }
```

```
    shader.fragmentShader = shader.fragmentShader.replace(
      target,
      /* glsl */ `
        vec3 viewDir = normalize(vViewPosition);
        float fresnel = pow(
          1.0 - clamp(dot(normalize(normal), viewDir), 0.0, 1.0),
          2.5
        );
        outgoingLight += vec3(0.45, 0.7, 1.0) * fresnel * 0.8;
        #include <opaque_fragment>
      `
    );
  };

  const sphere = new THREE.Mesh(geometry, material);
  group.add(sphere);

  /* ── Fine orbital loops (3 thin closed tubes, tilted so they never
     read as one flat ring). EllipseCurve.getPoints() repeats the
     endpoint — drop it, then close the CatmullRom curve explicitly. ── */
  const arcRadii = [7.8, 8.6, 9.4];
  arcRadii.forEach((r, i) => {
    const curve = new THREE.EllipseCurve(0, 0, r, r * 0.35, 0, Math.PI * 2, false, 0
    const points3D = curve
      .getPoints(80)
      .slice(0, -1) // remove duplicated closing point
      .map((p) => new THREE.Vector3(p.x, p.y, 0));
    const path = new THREE.CatmullRomCurve3(points3D, /* closed */ true);
    const tubeGeo = new THREE.TubeGeometry(path, 64, 0.018, 8, true);
    const arcMat = new THREE.MeshBasicMaterial({
      color: i === 1 ? 0x7CE0B8 : 0xB4DDFF,
      transparent: true,
      opacity: 0.55,
      depthWrite: false,
    });
    disposables.push(tubeGeo, arcMat);
    const loop = new THREE.Mesh(tubeGeo, arcMat);
    loop.rotation.x = Math.PI / 2 + i * 0.4;
    loop.rotation.z = i * 0.25;
    group.add(loop);
  });

  /* ── Particles: soft round points need a texture — a plain
     PointsMaterial renders squares. Small procedural radial sprite. ── */
  const sprite = (() => {
    const c = document.createElement('canvas');
    c.width = c.height = 64;
    const ctx = c.getContext('2d')!;
    const g = ctx.createRadialGradient(32, 32, 0, 32, 32, 32);
    g.addColorStop(0, 'rgba(255,255,255,1)');
    g.addColorStop(0.4, 'rgba(255,255,255,0.5)');
    g.addColorStop(1, 'rgba(255,255,255,0)');
    ctx.fillStyle = g;
    ctx.fillRect(0, 0, 64, 64);
    const tex = new THREE.CanvasTexture(c);
    return tex;
```

```
  })();
  disposables.push(sprite);

  const particleCount = 1800;
  const basePositions = new Float32Array(particleCount * 3);
  const seeds = new Float32Array(particleCount);
  for (let i = 0; i < particleCount; i++) {
    const phi = Math.random() * Math.PI * 2;
    const theta = Math.acos(2 * Math.random() - 1);
    const rad = 6.4 + Math.random() * 3.2;
    basePositions[i * 3] = rad * Math.sin(theta) * Math.cos(phi);
    basePositions[i * 3 + 1] = rad * Math.sin(theta) * Math.sin(phi);
    basePositions[i * 3 + 2] = rad * Math.cos(theta);
    seeds[i] = Math.random() * Math.PI * 2;
  }
  const pGeo = new THREE.BufferGeometry();
  pGeo.setAttribute('position', new THREE.BufferAttribute(basePositions.slice(), 3))
  const pMat = new THREE.PointsMaterial({
    map: sprite,
    color: 0xB4DDFF,
    size: 0.045,
    transparent: true,
    opacity: 0.85,
    blending: THREE.AdditiveBlending,
    depthWrite: false, // depth testing stays ON — only writes are disabled
  });
  disposables.push(pGeo, pMat);
  const points = new THREE.Points(pGeo, pMat);
  group.add(points);

  scene.add(group);

  /* ── Frame update: bounded drift around IMMUTABLE base positions.
     No accumulation, frame-rate independent (elapsed accumulator). ── */
  let elapsed = 0;
  const AMPLITUDE = 0.06;
  const update = (delta: number, reducedMotion: boolean) => {
    if (reducedMotion) return; // stable endpoints — nothing moves
    elapsed += Math.min(delta, 0.05); // clamp tab-switch deltas
    const pos = pGeo.attributes.position as THREE.BufferAttribute;
    for (let i = 0; i < particleCount; i++) {
      pos.setY(
        i,
        basePositions[i * 3 + 1] +
          Math.sin(elapsed * 0.6 + seeds[i]) * AMPLITUDE
      );
    }
    pos.needsUpdate = true;
    group.rotation.y += delta * 0.02; // slow ambient turn
  };

  const dispose = () => {
    scene.remove(group);
    disposables.forEach((d) => d.dispose());
  };
```

```
  return { group, material, points, update, dispose };
}
```

Integration — insert into the EXISTING render loop, do not replace it.
The project already owns a loop that drives camera presets, stations and the composer. Add one call inside it, and dispose on teardown:

```
const cerebro = createCerebroSphere(scene);

// inside the existing per-frame function, where delta is already computed:
cerebro.update(delta, useReducedMotionCurrent());

// on unmount / scene teardown:
cerebro.dispose();
```

Do not introduce `renderer.setAnimationLoop()` — it can drop the existing camera/station/composer updates.

Optional — audio-reactive rim: if the voice bridge already exposes a level meter, add `material.emissiveIntensity = 0.35 + level * 0.5;` in `update()`. Emissive must never be the only state signal.

Cool, not louder: the look comes from restrained depth — detached surfaces with consistent gaps, a blue/pink luminous interior, fine interrupted orbital accents, and a clear screen-stable assistant. Reuse the existing environment lighting; glass with nothing to refract reads as flat plastic.

3. Persistent Cerebro Widget (center overlay — full 5-axis contract)

```
// src/components/PersistentCerebroWidget.tsx
import React from 'react';
import { COLOUR_TOKENS } from '../tokens/colours';

export type ConnectionAxis = 'disconnected' | 'connecting' | 'connected' | 'reconnec
export type InputAxis = 'mic-off' | 'listening' | 'tentative' | 'final';
export type OutputAxis = 'silent' | 'speaking' | 'interrupted';
export type ExecutionAxis = 'idle' | 'validating' | 'waiting' | 'running' | 'failed'
export type PersistenceAxis = 'draft' | 'submitting' | 'applied' | 'conflict' | 'not

interface Props {
  connection: ConnectionAxis;
  inputState: InputAxis;
  outputState: OutputAxis;
  execution: ExecutionAxis;
  persistence: PersistenceAxis;
  actionLine: string;
  transcriptPreview: string;
  transcriptProvisional?: boolean;
  onMicToggle: () => void;
  onStop: () => void;
  onFullTranscript: () => void;
```

```
}

export const PersistentCerebroWidget: React.FC<Props> = ({
  connection,
  inputState,
  outputState,
  execution,
  persistence,
  actionLine,
  transcriptPreview,
  transcriptProvisional = true,
  onMicToggle,
  onStop,
  onFullTranscript,
}) => (
  <section className="persistent-widget" aria-label="Cerebro assistant state">
    {/* Avatar is the 3D sphere behind this frame; keep the frame translucent */}
    <div className="widget-head">
      {/* Connection is connectivity, NOT readiness — show both separately */}
      <span className="widget-title">CEREBRO / Connected</span>
      <span className="widget-connection">{connection}</span>
      <span className="widget-action-line">
        {actionLine || 'No active operation'}
      </span>
    </div>

    <div className="widget-controls">
      <button
        className="widget-btn"
        aria-pressed={inputState === 'listening'}
        onClick={onMicToggle}
      >
        Mic · {inputState}
      </button>
      <button className="widget-btn" onClick={onStop}>Stop action</button>
      <button className="widget-btn" onClick={onFullTranscript}>Full transcript</but
    </div>

    {transcriptPreview && (
      <div
        className="widget-transcript"
        aria-live="polite"
        data-provisional={transcriptProvisional}
      >
        LIVE TRANSCRIPT · {transcriptPreview}
        {transcriptProvisional && ' (provisional)'}
      </div>
    )}

    {/* All five axes as readable text — state must survive without
        sound, color or animation */}
    <div className="widget-state-grid" aria-label="Widget state axes">
      <span>connection: {connection}</span>
      <span>input: {inputState}</span>
      <span>output: {outputState}</span>
      <span>execution: {execution}</span>
```

```
      <span>persistence: {persistence}</span>
    </div>
  </section>
);
```

```
/* widget buttons — ≥44px touch targets (blueprint page 7) */
.widget-btn {
  min-height: 44px;
  padding: 10px 12px;
  background: transparent;
  border: 1px solid var(--divider-hairline, #3b4654);
  color: var(--text-primary, #f1eee8);
  border-radius: 8px;
  font-size: 12px;
  cursor: pointer;
}
```

Mount absolutely over the canvas at the same level as the existing overlays. The `persistence` axis comes from write receipts — a rendered form is still a draft. The default action line is `No active operation` unless an operation is genuinely underway.

4. Project Health Strip (bottom, always visible)

```
// src/components/ProjectHealthStrip.tsx — self-contained
import React from 'react';

export const ProjectHealthStrip: React.FC<{
  progress: string | 'unknown';
  blockers: string | 'unknown';
  due: string | 'unknown';
  onExpand: () => void;
}> = ({ progress, blockers, due, onExpand }) => (
  <footer className="health-strip" aria-label="Project health">
    <span><span aria-hidden="true">●</span> Progress: {progress}</span>
    <span><span aria-hidden="true">▲</span> Blockers: {blockers}</span>
    <span><span aria-hidden="true">☒</span> Due: {due}</span>
    <button className="widget-btn" onClick={onExpand}>Project Health</button>
  </footer>
);
```

Text plus color, never color alone. Show `'unknown'` whenever the source value is missing — never invent progress or receipts (blueprint evidence rules). Drive values from the existing `PROJECTS_DATA` / health logic.

5. Responsive Layout Shell

The blueprint's 1024×768 coordinates are a **reference composition**, not fixed coordinates for every tablet. Use a gap-based grid that degrades cleanly; all positioning via logical properties so RTL mirrors correctly.

```
/* src/styles/cerebro-shell.css */
.cerebro-shell {
  position: relative;
  width: 100%;
  height: 100dvh;                    /* dynamic viewport — mobile chrome safe */
  background: #101D27;
  overflow: hidden;
  display: grid;
  grid-template-areas:
    "world widget surface"
    "health health health";
  grid-template-columns: 28% 38% 28%;
  grid-template-rows: 1fr auto;
  gap: 14px;                         /* ONE gap constant — separation knob */
  padding: 16px 32px
           calc(16px + env(safe-area-inset-bottom));
}
.world-left   { grid-area: world; }
.widget-center{ grid-area: widget; align-self: center; }
.surface-right{ grid-area: surface; overflow-y: auto; } /* scrollable surface */
.health-strip{ grid-area: health; min-height: 48px; }   /* 44px button fits */

/* Reference composition near 1024×768 — adjust columns, don't hard-code
   the blueprint's pixel coordinates across a whole range */
@media (min-width: 820px) and (max-width: 1200px) {
  .cerebro-shell { grid-template-columns: 27% 1fr 27%; padding: 16px 20px; }
}

/* Phone: stacked — compact widget, sheet below, health above input.
   Avatar stays visible while the sheet scrolls (blueprint page 7). */
@media (max-width: 819px) {
  .cerebro-shell {
    grid-template-areas: "widget" "world" "surface" "health";
    grid-template-columns: 1fr;
    grid-template-rows: auto 1fr auto auto;
    padding: 16px
             16px
             calc(16px + env(safe-area-inset-bottom));
  }
  .persistent-widget { min-height: 148px; max-height: 184px; }
  .widget-transcript { max-height: 72px; overflow-y: auto; font-size: 12px; }
  .health-strip { font-size: 12px; }  /* never below 12px (blueprint page 7) */
}

@media (prefers-reduced-motion: reduce) {
  .cerebro-shell * { transition: none !important; animation: none !important; }
}
```

Use the existing `useViewport` to switch compositions; the fixed-height widget keeps its controls and state text unclipped (no `overflow: hidden` on the widget itself).

6. Bloom and performance

- **DPR cap:** `renderer.setPixelRatio(Math.min(1.5, window.devicePixelRatio))` — apply inside the existing renderer init/resize logic, and resize the composer's render targets consistently. This is a proposed initial budget, not a measured guarantee.
- **Threshold-based bloom:** keep the existing composer and bloom pass. A luminance threshold above typical scene brightness means only genuinely bright pixels (additive particles, the Fresnel rim, the emissive core) bloom — but the threshold affects *any* bright pixel, so tune against the real scene: start near `threshold: 0.85, strength: 0.6, radius: 0.5` and adjust while watching a mid-toned station panel. Verify the composer's HDR/output tone-mapping configuration before changing numbers. If true per-object selectivity is required later, that needs a layer-masked bloom, not a threshold.
- **Context loss:** keep the existing handler; the DOM overlays (widget, health strip) must remain usable when the canvas dies.

7. Integration Steps (order of implementation)

1. **Sphere upgrade** — insert `createCerebroSphere` into the existing scene init; add `update(delta, reducedMotion)` inside the existing loop; `dispose()` on teardown. Verify the installed three version supports `<opaque_fragment>` (r154+).
2. **Widget component** — add `PersistentCerebroWidget`; wire to the existing voice/Gemini state; the persistence axis reads from write receipts.
3. **Health strip** — extract/enhance the existing health UI into the bottom strip; `'unknown'` when the source is missing.
4. **Stations** — use the elevated camera presets already in the project; label via the existing anchored-label mechanism.
5. **Right surface** — reuse `PanelsOverlay` / `WireframesV11Panels` content constrained to the right region; scrollable.
6. **Reduced motion** — pass the *current* preference into `update()` each frame; freeze drift, ambient turn and camera auto-orbit.
7. **DPR & cleanup** — cap pixel ratio at 1.5 inside the existing resize logic; dispose geometries/materials/textures on unmount so repeated mounts don't leak.

8. Google AI Studio Prompt (use after pasting the above)

```
Upgrade the Cerebro GUI to match the V2.0 blueprint shared screen.
This is an existing project — do not rebuild or migrate it. First verify
the installed three.js version, the actual component paths, the renderer
and the composer configuration, then adapt the provided code to them.
Work in the order below and stop after each step to report what changed.
```

```
1. SPHERE: extract createCerebroSphere (code provided) into the scene
   init. MeshPhysicalMaterial transmission 0.92, thickness 1.4, ior 1.45,
   clearcoat 0.6, attenuationColor #4C8DFF. Fresnel rim patched via
   onBeforeCompile replacing the <opaque_fragment> include (three r154+
   name) with a guard that warns if the chunk is missing. Three closed
   TubeGeometry orbital loops (radii 7.8/8.6/9.4, tilted, depthWrite
   false) and 1800 additive soft-textured Points on a 6.4-9.6 shell with
   BOUNDED drift around immutable base positions. Expose
   update(delta, reducedMotion) and dispose(). Call update() inside the
   existing render loop — do not replace the loop.

2. WIDGET: add PersistentCerebroWidget (code provided) as a screen-stable
   center overlay with ALL FIVE state axes as text: connection, input,
   output, execution, persistence. Connection and readiness are separate.
   Default action line: "No active operation". Controls: Mic, Stop action,
   Full transcript, each ≥44px. State must be readable without sound,
   color or animation. Persistence reads from write receipts — a rendered
   form is still a draft.

3. HEALTH STRIP: compact bottom strip, min-height 48px, Progress/Blockers/
   Due as text plus color; show "unknown" whenever the source value is
   missing. Never invent progress or receipts.

4. LAYOUT: gap-based grid — left stations / center widget / right
   scrollable surface / bottom health. Treat the blueprint's 1024×768
   coordinates as a reference composition, not fixed coordinates. Phone
   stacks: 148-184px widget, 40px-class health strip (48px so 44px buttons
   fit), safe-area insets, 100dvh, text never below 12px. Positioning via
   logical properties so Arabic RTL mirrors correctly.

5. PERFORMANCE: cap DPR at 1.5 inside the existing resize logic and
   resize composer targets consistently. Keep the existing bloom pass —
   tune its threshold against the real scene (start 0.85) so only bright
   pixels bloom; this is threshold-based, not true per-object selectivity.
   prefers-reduced-motion freezes particle drift, ambient turn and camera
   auto-orbit — stable endpoints only. Call dispose() on unmount.

NON-NEGOTIABLES
- Preserve camera presets, stations, voice bridges, bloom, data layer,
  routes and auth. No behavior changes.
- Keep English and Arabic equally correct (RTL via logical properties).
- No new libraries; adapt the provided code, do not rewrite the app.
- Do not claim tests ran unless they ran. Report changed files honestly.
```

Scope note: this covers the shared screen and its visual effects — not all 60 blueprint screens. Start with the sphere + widget; the shell locks into place around them, and every step is reversible.